Sri Lanka Institute of Information Technology

Current Trends in Software Engineering (SE4010) | 2026 | Semester 1

ASSIGNMENT 01

eBuy — Cloud-Native E-Commerce Platform

Microservices on AWS ECS (Fargate)

Git Repo : https://github.com/amila1998/eBuy-Cloud-Market

Live Demo: http://ebuy-load-balancer-789736785.us-east-1.elb.amazonaws.com/

Submitted by

Name	REGISTRATION NUMBER	Status	Batch	Group ID
Senarathne S M A D	IT20089436	Pro-rata	Weekend	61

Contents

eBuy — Cloud-Native E-Commerce Platform 1

Introduction 4

Shared Architecture Diagram 5

Overview 5

Architectural Diagram 6

Communication Flow Summary 7

Auth Service — Description and Rationale 7

Role in the System 7

What It Does 7

Design Rationale 8

Inter-Service Communication 8

Synchronous — JWT Verification (Auth → All Services) 8

Synchronous HTTP — Order Service → Product Service 8

Asynchronous Kafka Events — Order Service ↔ Payment Service 8

DevOps and Security Practices 9

CI/CD Pipeline (GitHub Actions) 9

Containerization 9

Secret Management 10

Security Controls 10

Logging and Observability 10

Challenges and How They Were Addressed 11

Redis Connection Refused in ECS (`ECONNREFUSED 127.0.0.1:6379`) 11

Docker Build Failing — Missing `public/` Directory 11

CI npm Cache Failure — Missing Lock Files 11

Service Discovery in ECS 12

Appendix 12

Product Service verifying a JWT from Auth Service 12

Request from Order Service 13

Product Service Response (stock available) 13

Product Service Response (insufficient stock) 13

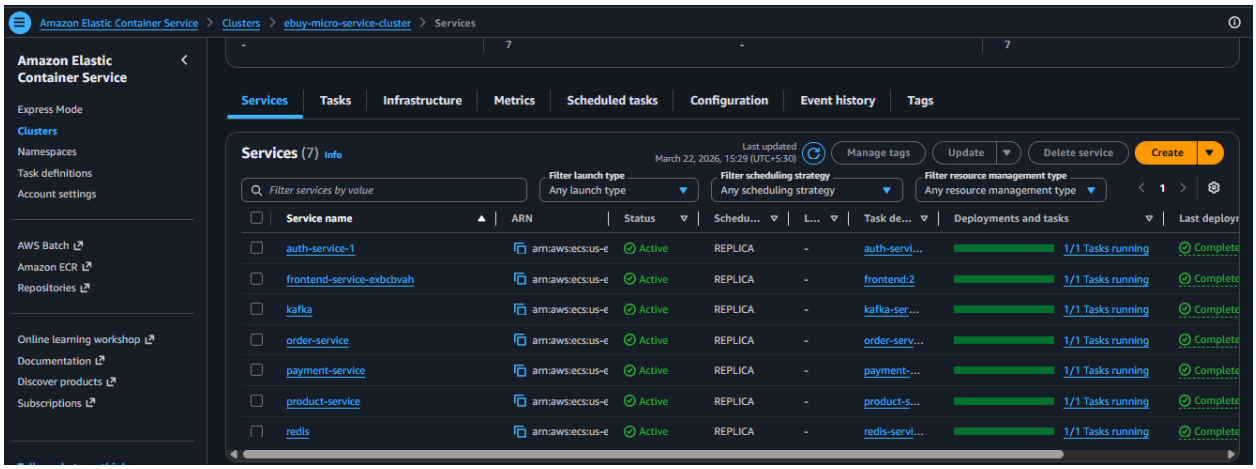
Order Service publishes `order.created` 14

Payment Service consumes `order.created`, processes payment (90% simulated success), then publishes result 14

Order Service consumes `payment.success` / `payment.failed` and updates order status 14

GIthub Action 15

AWS Cluster



..... 15

AWS Task definitions 16

AWS Load Balancer 16

Introduction

Modern application development has shifted from monolithic architectures to microservices-based systems, driven by the need for scalability, flexibility, and independent deployment. Microservices enable applications to be broken down into smaller, loosely coupled services that can be developed, deployed, and scaled independently. This approach is especially suitable for cloud environments, where managed infrastructure and automation tools support rapid development and continuous delivery.

This project presents the design and implementation of eBuy, a cloud-native e-commerce platform built using a microservices architecture. The system consists of four core backend services—Auth Service, Product Service, Order Service, and Payment Service—along with a Next.js frontend, all deployed on Amazon Web Services (AWS) using Elastic Container Service (ECS) with Fargate. Each microservice is containerized using Docker and deployed independently, ensuring modularity and ease of maintenance.

The platform demonstrates real-world software engineering practices by integrating:

- Synchronous communication via REST APIs for immediate data exchange
- Asynchronous communication using Apache Kafka for event-driven workflows
- Centralized data storage using Redis for fast and efficient data access

In addition to functional requirements, this project emphasizes DevOps and DevSecOps practices. A fully automated CI/CD pipeline using GitHub Actions is implemented to handle testing, building, containerization, and deployment. Security best practices such as JWT-based authentication, password hashing, environment-based secret management, and least-privilege access control are incorporated to ensure secure system operation.

The objective of this project is to demonstrate the practical implementation of:

- Microservices architecture in a cloud environment
- Containerization and orchestration using AWS services
- Inter-service communication patterns (REST and event-driven)
- DevOps automation and secure application deployment

Overall, the eBuy system provides a comprehensive prototype that reflects modern cloud-native application design, combining scalability, reliability, and security within a cohesive microservices ecosystem.

Shared Architecture Diagram

The eBuy system is designed using a **microservices architecture**, where the application is divided into independent services that communicate with each other to deliver complete business functionality. The architecture follows a **cloud-native approach**, leveraging managed services in AWS to ensure scalability, reliability, and ease of deployment.

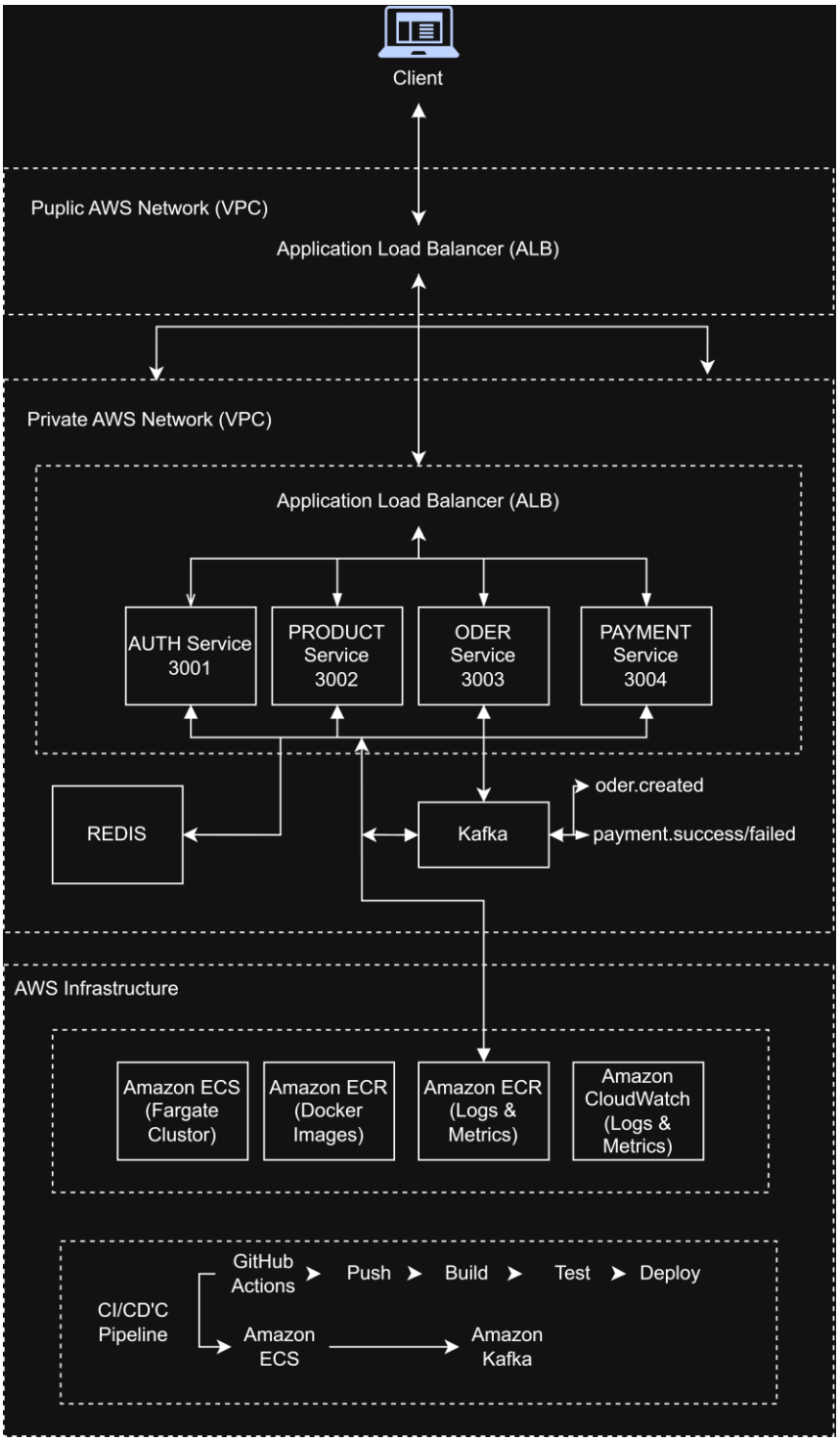
Overview

The system consists of the following main components:

- **Frontend (Next.js)**
  The user interface of the application, responsible for interacting with users and sending requests to backend services via a public endpoint.
- **API Access Layer (Application Load Balancer)**
  Acts as the entry point to the system. It routes incoming HTTP requests from users to the appropriate backend microservices based on URL paths.
- **Backend Microservices**
  - **Auth Service** – Handles user authentication, registration, and JWT token generation.
  - **Product Service** – Manages product catalog, including product creation, retrieval, and updates.
  - **Order Service** – Processes customer orders and coordinates with other services.
  - **Payment Service** – Simulates payment processing and emits payment-related events.
- **Message Broker (Kafka)**
  Enables asynchronous, event-driven communication between services. For example:
  - Order Service publishes *order.created* events
  - Payment Service consumes these events and publishes *payment.success* or *payment.failed*
- **Data Store (Redis)**
  Used as a fast in-memory database for caching, session management, and storing temporary data.
- **Container Orchestration (AWS ECS Fargate)**
  Each microservice is containerized using Docker and deployed as an independent service on ECS Fargate, enabling serverless container management.
- **Container Registry (Amazon ECR)**
  Stores Docker images for all services, which are pulled during deployment.
- **CI/CD Pipeline (GitHub Actions)**
  Automates the build, test, and deployment process for all microservices, ensuring continuous integration and delivery.

Architectural Diagram

The diagram below illustrates the overall architecture of the system:



Communication Flow Summary

Direction	Protocol	Purpose
Frontend → Auth Service	HTTP REST	User registration, login, token verification
Frontend → Product Service	HTTP REST	Browse, create, update, delete products
Frontend → Order Service	HTTP REST	Place and retrieve orders
Frontend → Payment Service	HTTP REST	View payment status
Order Service → Product Service	HTTP REST (sync)	Stock availability check before order creation
Order Service → Kafka	Event (`order.created`)	Triggers payment processing asynchronously
Payment Service → Kafka	Event (`payment.success` / `payment.failed`)	Notifies order service of payment outcome
All Services → Redis	TCP	Persistent data store (users, products, orders, payments)

Auth Service — Description and Rationale

Role in the System

The **Auth Service** is the identity and access management gateway for the entire eBuy platform. Every user interaction that requires authentication is routed through this service. It is the first service a user encounters — handling account creation and login — and its issued JWT tokens are the credentials consumed by all other services to authorise requests.

What It Does

Endpoint	**Method**	**Description**
`/api/user/register`	POST	Creates a new user account, stores hashed credentials in Redis, returns a JWT
`/api/user/login`	POST	Validates credentials against Redis, returns a JWT on success
`/api/user/me`	GET	Returns the currently authenticated user's profile (protected)
`/health`	GET	Health check endpoint for ECS task health monitoring

Design Rationale

Fastify was chosen over Express for its performance and built-in schema validation. The route schema for `/register` enforces `email` format and a minimum password length of 6 characters at the framework level, rejecting malformed requests before they reach application code.

Redis is used as the user store rather than a relational database because:

- The data model is simple (one hash key per user: `user:{email}`)
- It provides sub-millisecond lookups by email key
- The same Redis instance is shared across services, avoiding infrastructure duplication in a student-scale deployment

JWT (JSON Web Tokens) with a 7-day expiry are issued on both registration and login. The secret is injected at runtime via the `JWT_SECRET` environment variable — never hardcoded — and the same secret is configured on all other services so they can verify tokens independently without calling back to the auth service on every request (stateless verification).

bcryptjs with 12 salt rounds is used for password hashing. This cost factor was chosen to balance security (resistance to brute force) with response time on Fargate's 1 vCPU allocation.

Inter-Service Communication

Synchronous — JWT Verification (Auth → All Services)

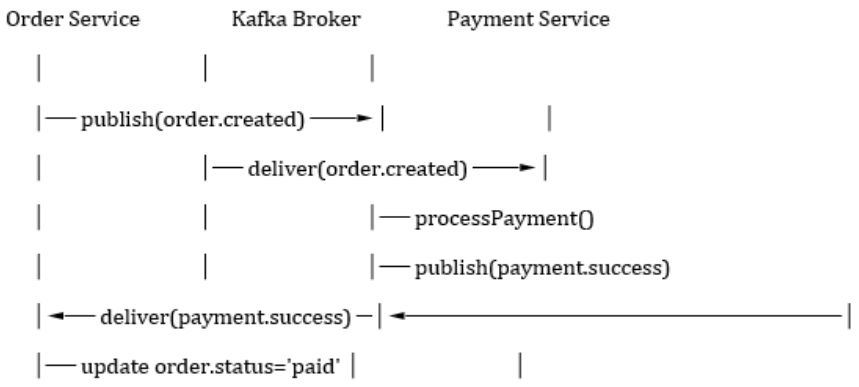
The auth service does not expose a token verification endpoint. Instead, all services share the same `JWT_SECRET` and verify tokens locally using `@fastify/jwt`. This eliminates a single point of failure and a network hop on every request.

Synchronous HTTP — Order Service → Product Service

When a user places an order, the order service makes a synchronous HTTP call to the product service to verify stock before committing the order.

Asynchronous Kafka Events — Order Service ↔ Payment Service

The order and payment services communicate asynchronously through Apache Kafka (KRaft mode, no ZooKeeper).



DevOps and Security Practices

CI/CD Pipeline (GitHub Actions)

The pipeline in `.github/workflows/ci-cd.yml` enforces a linear quality gate:

```
push to main
  |
  ├── test-auth    ──┐
  ├── test-product ──┤── All must pass
  ├── test-order   ──┤
  └── test-payment ──┘
      |
      ▼
  build-and-push  (only on main push)
  ├── docker build each service
  ├── tag with git SHA (immutable) + latest
  └── push to Amazon ECR
      |
      ▼
  deploy (only on main push)
  └── aws ecs update-service --force-new-deployment
```

Key practices:

- **Immutable image tags** — each image is tagged with the full git commit SHA (`github.sha`), ensuring every deployment is fully traceable and rollbacks are trivial
- **Test gate** — no image is built unless all four service test suites pass
- **Parallel testing** — all four service test jobs run concurrently, reducing pipeline time
- **npm lock files** — `package-lock.json` is committed for each service, ensuring `npm ci` reproducibility and enabling GitHub Actions npm caching

Containerization

Each service has a dedicated `Dockerfile` with:

- `node:20-alpine` base image (minimal attack surface)
- `npm install --production` (dev dependencies excluded from the image)
- `EXPOSE` declaration for the correct port
- Non-root process via Node process model

The frontend uses a **multi-stage Dockerfile**:

1. **Builder stage**— installs all deps, runs `next build` with `output: 'standalone'`
2. **Runner stage** — copies only the standalone output (no `node_modules`), producing a significantly smaller final image

Secret Management

All sensitive configuration is handled as environment variables injected at runtime.

Secret	Where used
JWT_SECRET	Auth, Product, Order, Payment services (token signing/verification)
REDIS_URL	All backend services (Redis connection string including password)
REDIS_PASSWORD	All backend services
KAFKA_BROKERS	Order and Payment Services
PRODUCT_SERVICE_URL	Order Service
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY	CI/CD pipeline only

Secrets are stored in GitHub Actions Secrets and Some in Aws Task Definitions and never appear in source code or Docker images. The Redis password is embedded in the `REDIS_URL` (`redis://:password@host:6379`) so the application doesn't need to separately manage authentication.

Security Controls

Control	Implementation
Password hashing	`bcryptjs` with 12 salt rounds (OWASP-recommended cost factor)
Authentication	Stateless JWT, 7-day expiry, HS256 algorithm
Input validation	Fastify JSON Schema on all request bodies (type, format, required fields)
Authorization	`authenticate` decorator applied to all protected routes; order ownership validated on retrieval
CORS	Explicitly configured on all services via `@fastify/cors`
Injection protection	No raw query construction — all data access via Redis client methods with key scoping

Logging and Observability

- **Fastify logger** enabled on all services — structured JSON logs including request method, URL, status, and response time

- **AWS CloudWatch Logs** — each service writes to a dedicated log group (`/ecs/{service-name}`) configured via the ECS task definition's `awslogs` log driver with `awslogs-create-group: true` (group auto-created)
- **Health endpoints** — every service exposes `GET /health` returning `{ status: 'ok', service: '<name>' }`, used by ECS health checks to determine task health before routing traffic

Challenges and How They Were Addressed

Redis Connection Refused in ECS (`ECONNREFUSED 127.0.0.1:6379`)

Challenge: After deploying to ECS Fargate, all services logged `Redis client error: connect ECONNREFUSED 127.0.0.1:6379`. The docker-compose setup worked locally, but ECS containers failed to connect.

Root cause: ECS task definitions did not have the `REDIS_URL` environment variable set. The application therefore used the hardcoded fallback `redis://localhost:6379`, which doesn't exist within a container — Redis runs as a separate ECS task accessible only by its service discovery DNS name.

Resolution:

1. Updated the CI/CD deploy step to call `aws ecs register-task-definition` before every deployment, injecting the correct `REDIS_URL` (`redis://:password@redis-service-service-mqa7acta:6379`) from GitHub Secrets into the task definition
2. Ensured `REDIS_URL` and all other service-discovery URLs are passed as ECS task-level environment variables on every deploy, not just at initial setup

Docker Build Failing — Missing `public/` Directory

Challenge: The frontend Docker build failed in CI with:

ERROR: "/app/public": not found

The Dockerfile's runner stage copied `COPY --from=builder /app/public ./public`, but no `public/` directory existed in the repository.

Resolution: Created `frontend/public/.gitkeep` to materialise the directory in git. Since `COPY . .` in the builder stage copies everything from the repo (including the now-present empty `public/` directory) into `/app`, the runner stage's `COPY --from=builder /app/public ./public` succeeds.

CI npm Cache Failure — Missing Lock Files

Challenge: All four CI test jobs failed with:

Error: Some specified paths were not resolved, unable to cache dependencies.

`actions/setup-node` was configured with `cache: 'npm'` and `cache-dependency-path: <service>/package-lock.json`, but no `package-lock.json` files were committed.

Resolution: Generated `package-lock.json` for all services locally using `npm install --package-lock-only --ignore-scripts` and committed them. This also ensures `npm ci` (which strictly requires a lock file) works correctly in CI.

Asynchronous Payment Integration

Challenge: Designing the payment flow without creating tight coupling between order and payment services. A synchronous HTTP call from order service to payment service would mean an order can't be created if payment service is down.

Resolution: Adopted an event-driven pattern using Kafka:

- Order service publishes `order.created` and immediately returns the order to the client with status `pending`
- Payment service consumes the event asynchronously, processes payment, and publishes `payment.success` or `payment.failed`
- Order service consumes the payment result and updates the order status

This decouples the two services entirely. A payment service restart or slowdown does not affect order creation, and the system naturally retains messages in Kafka until they are consumed.

Service Discovery in ECS

Challenge: In local docker-compose, services reference each other by container name (e.g., `http://product-service:3002`). In ECS, each service gets an AWS Cloud Map DNS entry that differs from the container name.

Resolution: Internal service URLs and Kafka broker addresses are injected as environment variables (`PRODUCT_SERVICE_URL`, `KAFKA_BROKERS`) in the ECS task definitions. The application code uses these variables with sensible localhost fallbacks for local development:

```
const PRODUCT_SERVICE_URL = process.env.PRODUCT_SERVICE_URL || 'http://localhost:3002';
```

This keeps local and cloud environments compatible without code changes.

Appendix

Product Service verifying a JWT from Auth Service

// product-service/src/index.js

```
app.register(fastifyJwt, {
  secret: process.env.JWT_SECRET,
});

app.decorate('authenticate', async (request, reply) => {
  try {
    await request.jwtVerify();  // validates signature using shared secret
  } catch (err) {
    reply.code(401).send({ error: 'Unauthorized' });
  }
});
```

A user logs in via the auth service, receives a token, then sends it as `Authorization: Bearer <token>` to the product service — which validates it without any call back to the auth service.

Request from Order Service

```
POST http://product-service-service-s4aqsu66:3002/api/products/{productId}/check-stock
Content-Type: application/json
{ "quantity": 2 }
```

Product Service Response (stock available)

```
{ "available": true, "product": { "name": "Laptop", "price": 999.99, "stock": 10 } }
```

Product Service Response (insufficient stock)

```
{ "available": false, "stock": 1 }
```

The order service uses the product details returned (name, price) to compute the `totalAmount` and embed product metadata directly in the order record, avoiding future cross-service lookups.

```
// order-service/src/controllers/orderController.js
const res = await fetch(`${PRODUCT_SERVICE_URL}/api/products/${productId}/check-stock`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
```

```
  body: JSON.stringify({ quantity }),
});
const data = await res.json();
if (!res.ok || !data.available) {
  return reply.code(400).send({ error: 'Insufficient stock', details: data });
}
```

Order Service publishes `order.created`

```
// order-service/src/plugins/kafka.js
await publishEvent('order.created', {
  orderId: order.id,
  userId: order.userId,
  productId,
  quantity,
  totalAmount: order.totalAmount,
});
```

Payment Service consumes `order.created`, processes payment (90% simulated success), then publishes result

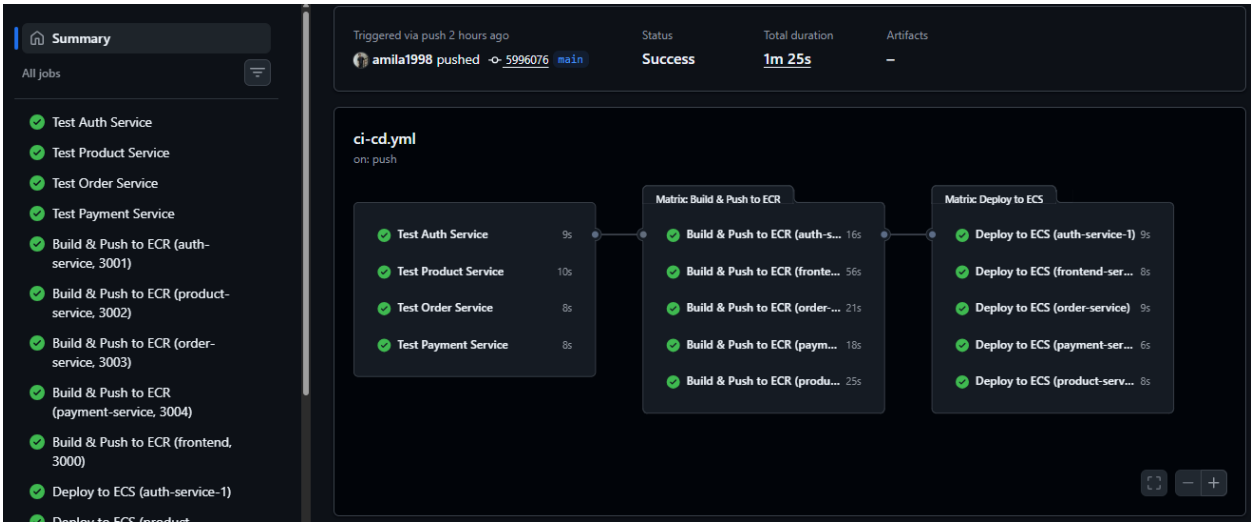
```
// payment-service/src/plugins/kafka.js
const success = Math.random() < 0.9;
const topic = success ? 'payment.success' : 'payment.failed';
await publishEvent(topic, {
  paymentId, orderId, userId, amount, status: payment.status,
});
```

Order Service consumes `payment.success` / `payment.failed` and updates order status

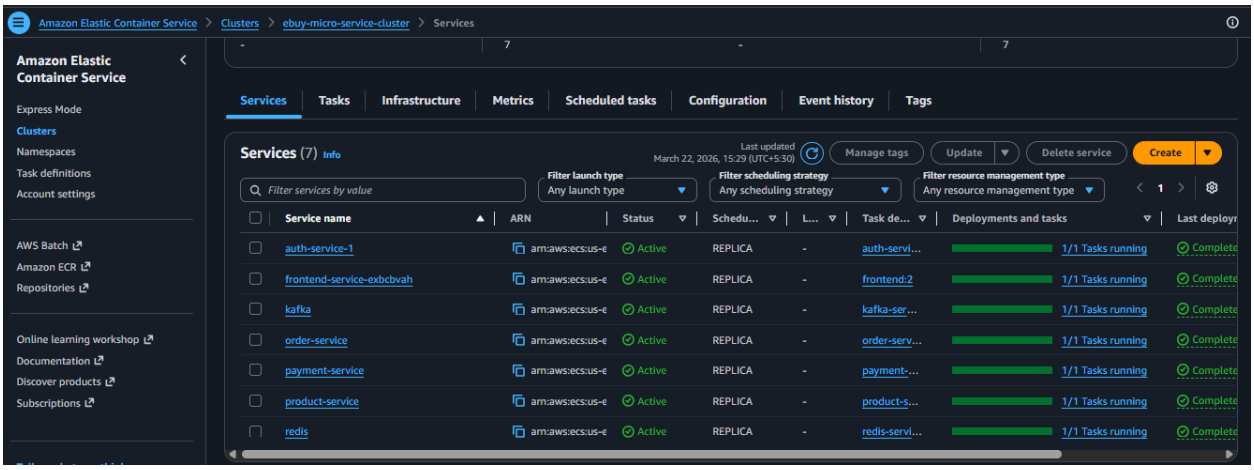
```
order.status = topic === 'payment.success' ? 'paid' : 'payment_failed';
await redis.set(`order:${order.id}`, JSON.stringify(order));
```

This event-driven pattern means the payment service can be scaled, restarted, or temporarily unavailable without causing the order service to fail or block.

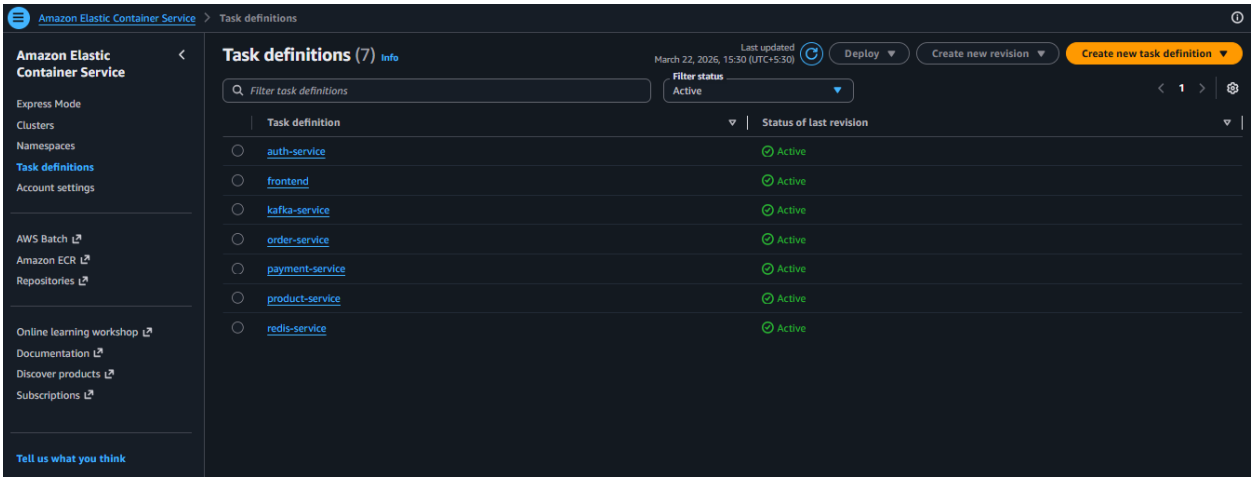
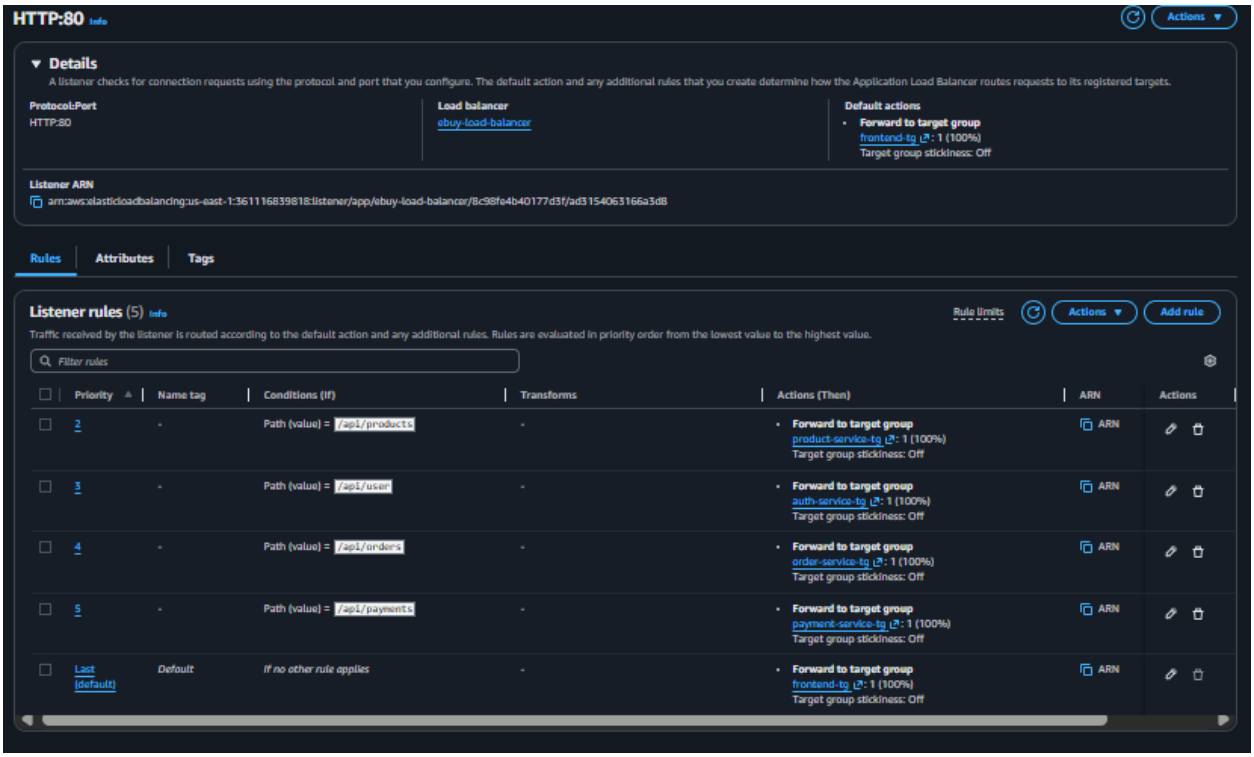
GIthub Action



AWS Cluster



AWS Task definitions



AWS Load Balancer